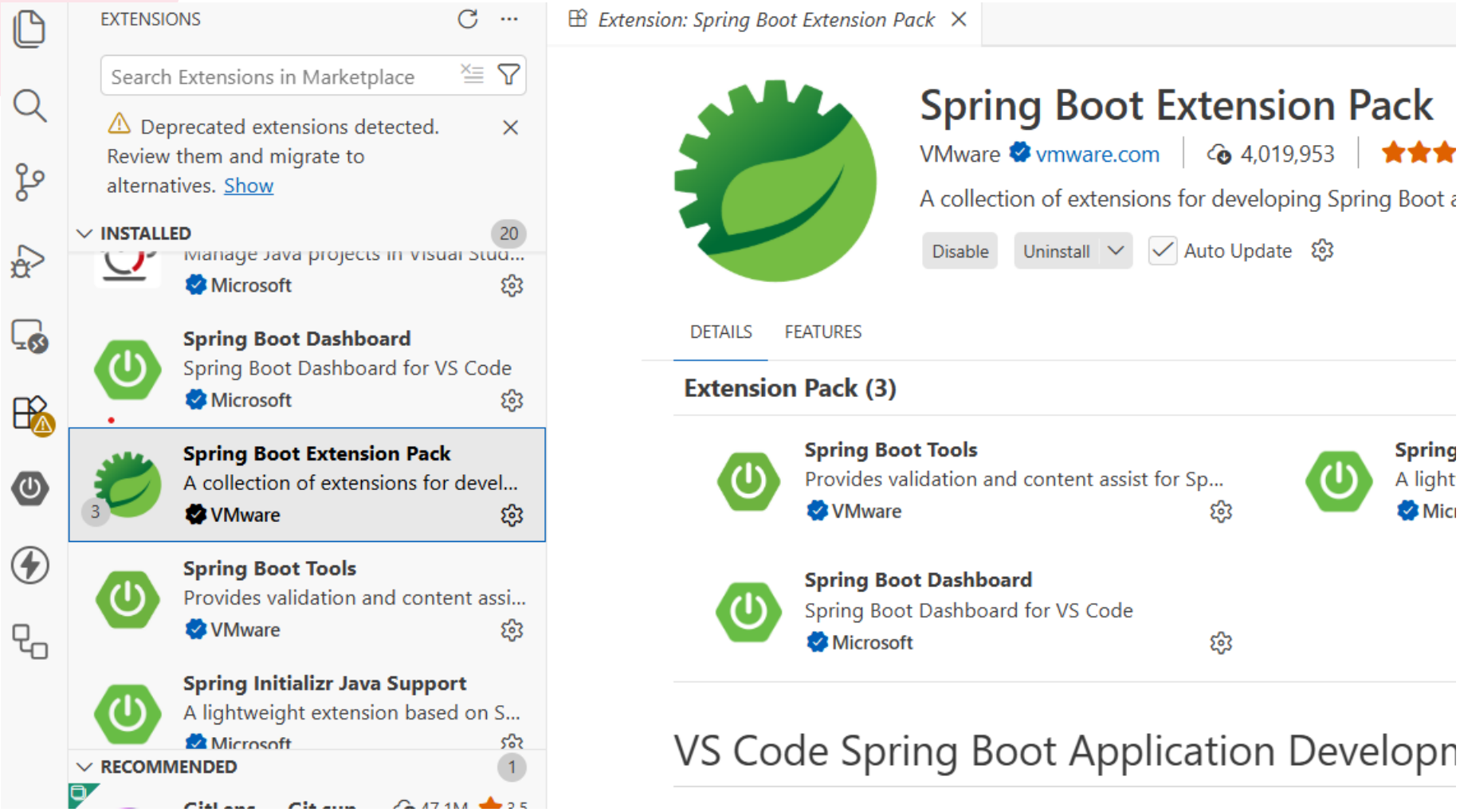
Spring Boot

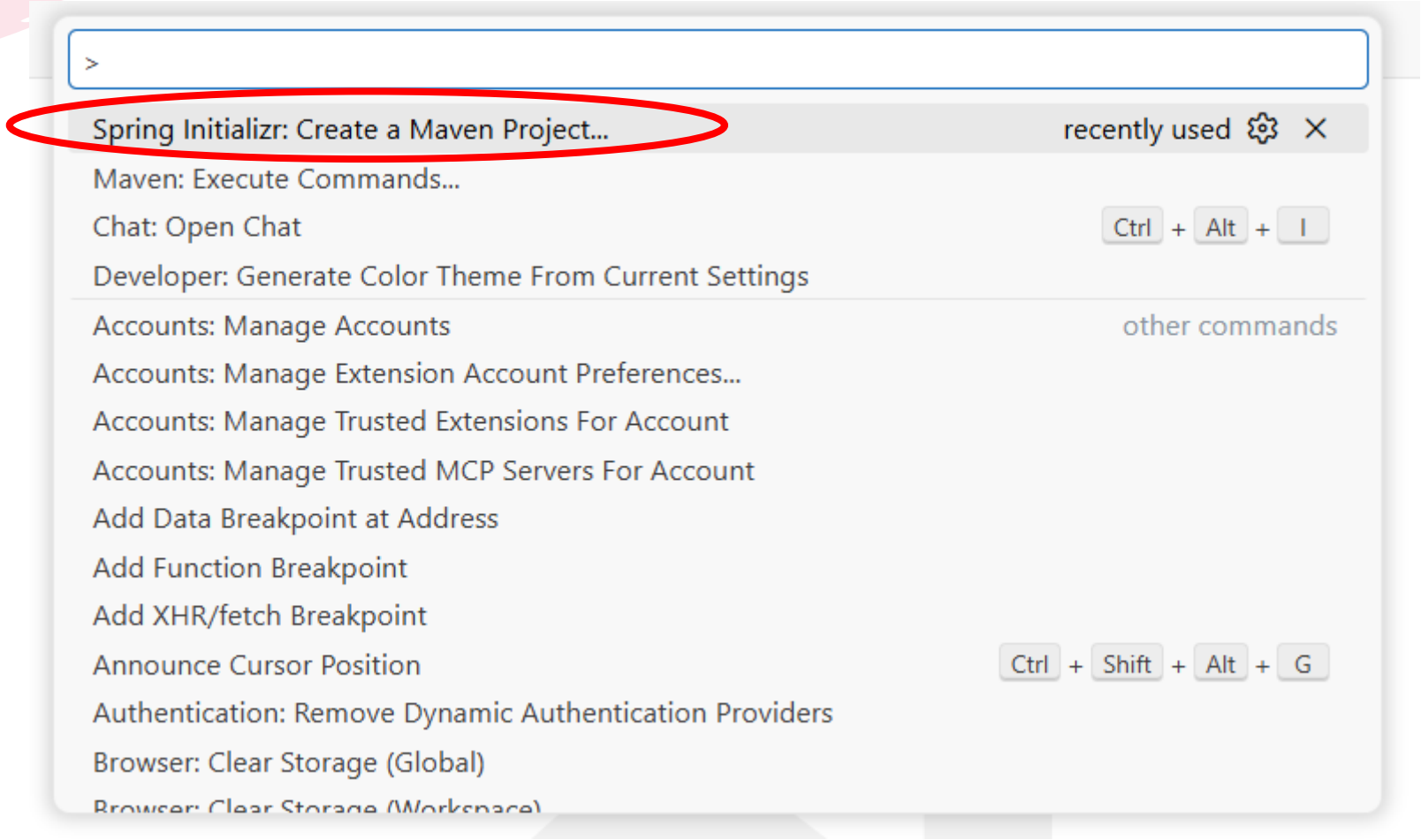
Introdução

PAULO R T CÂNDIDO

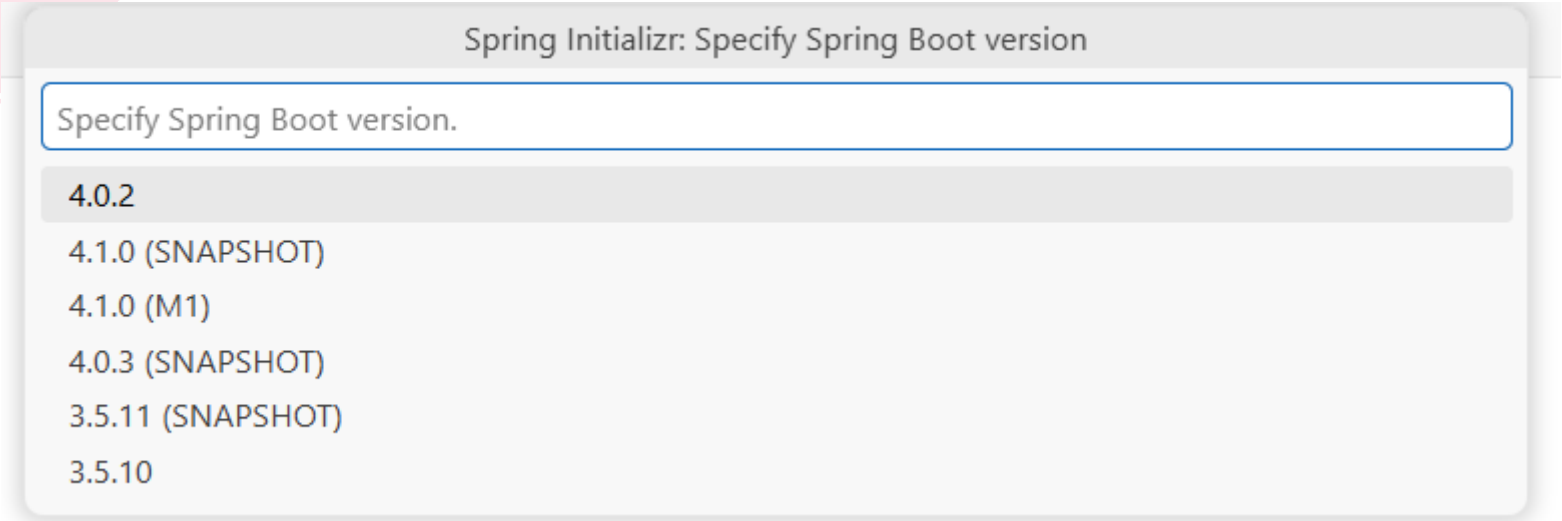
VSC - Spring Boot Extension Pack



Criar Projeto



Versões

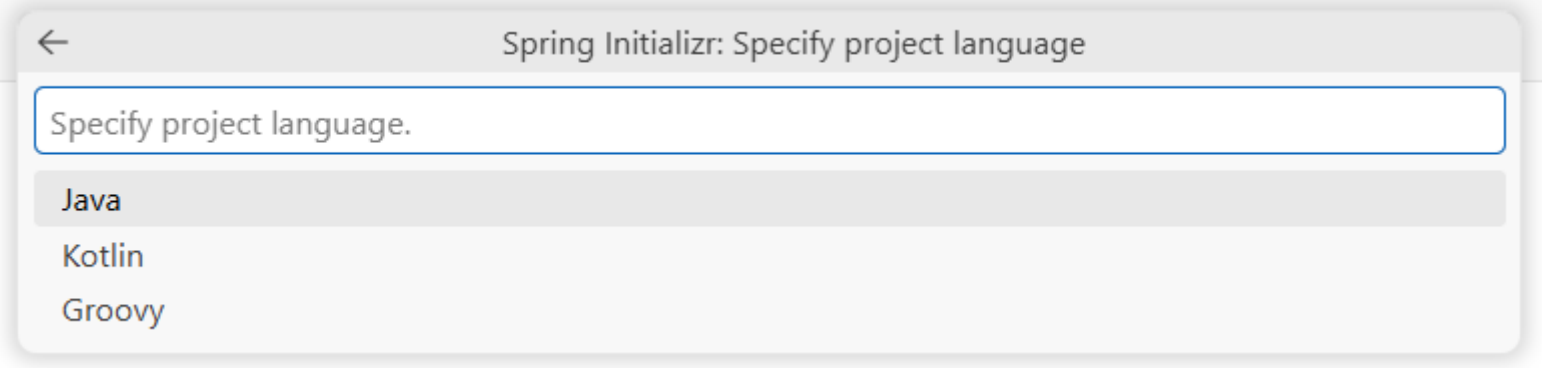


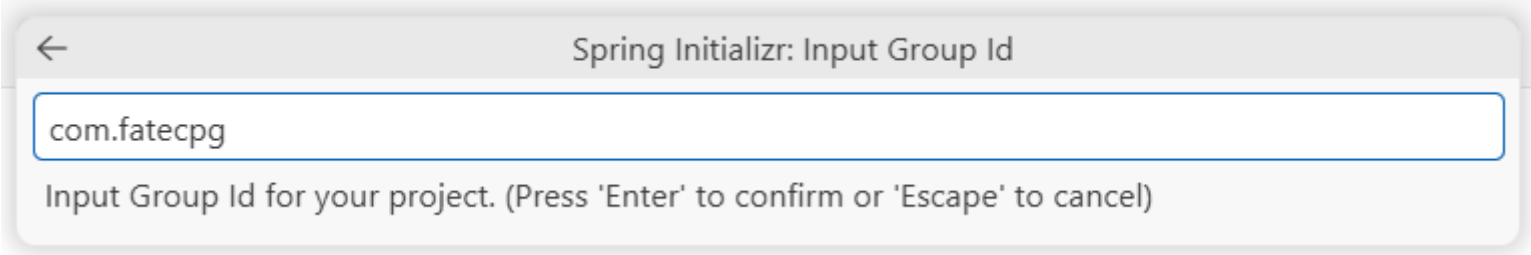
Escolha a versão "GA" (General Availability)

Sempre escolha as versões que **não possuem sufixo** (como Snapshot, Milestone ou RC).

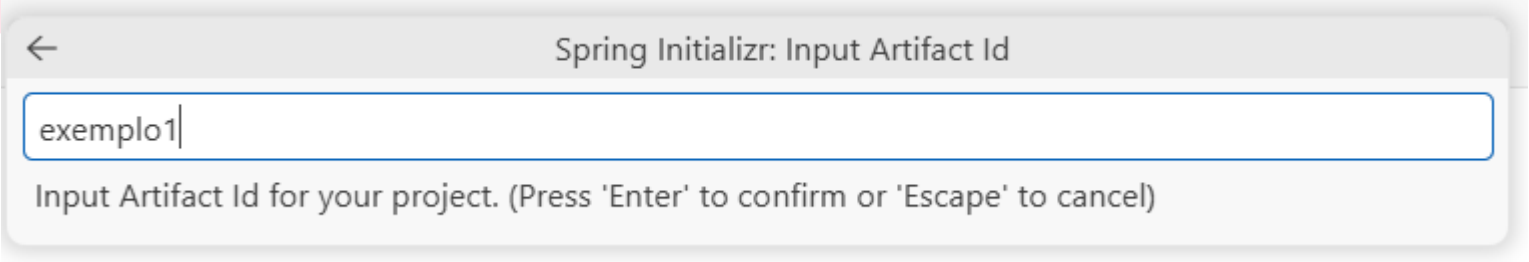
- **Snapshot:** Versão em desenvolvimento, instável. Fuja dela para projetos sérios.
- **M (Milestone) / RC (Release Candidate):** Versões de teste antes do lançamento oficial.
- **GA (ou sem sufixo):** É a versão estável, recomendada para uso real.

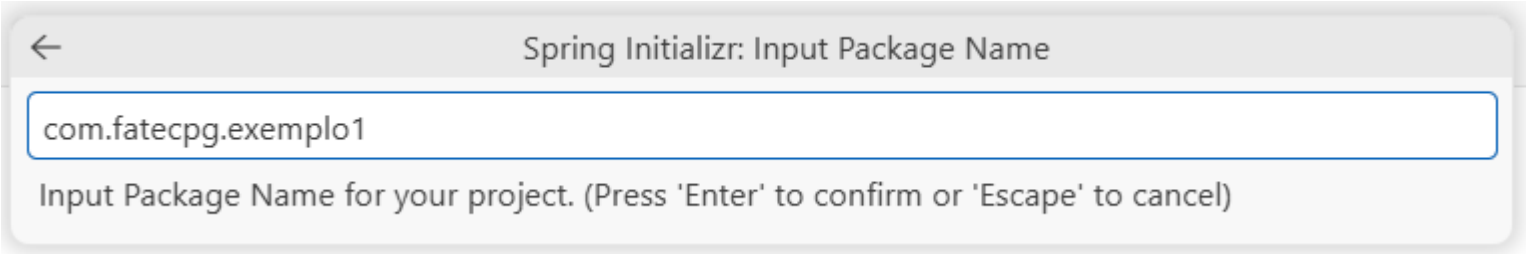
Linguagem e Group Id

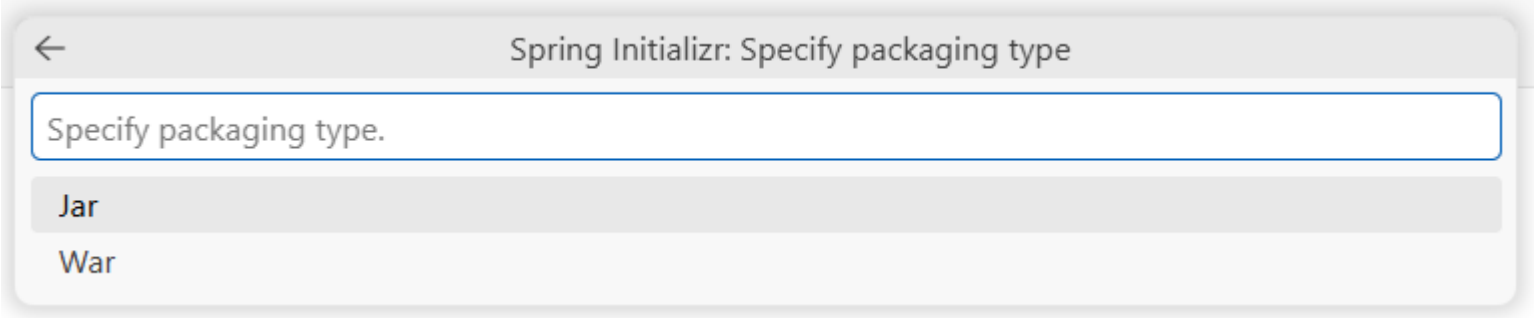




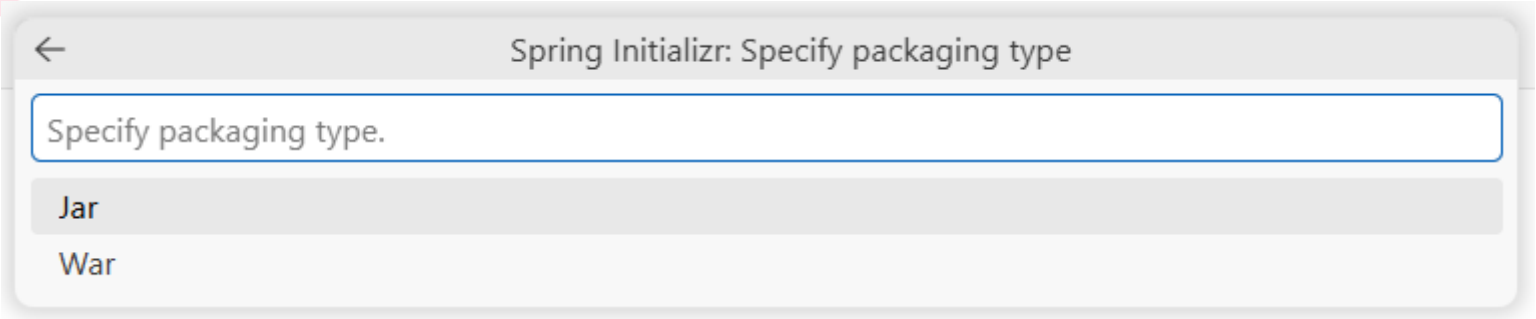
Nome do projeto e pacote (eq namespace)





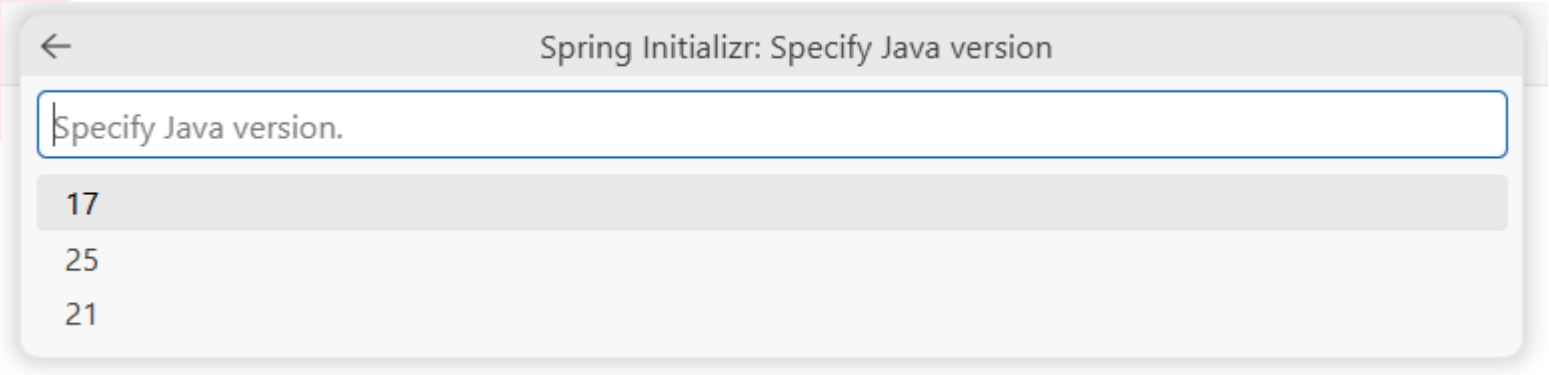


Compactação para distribuição

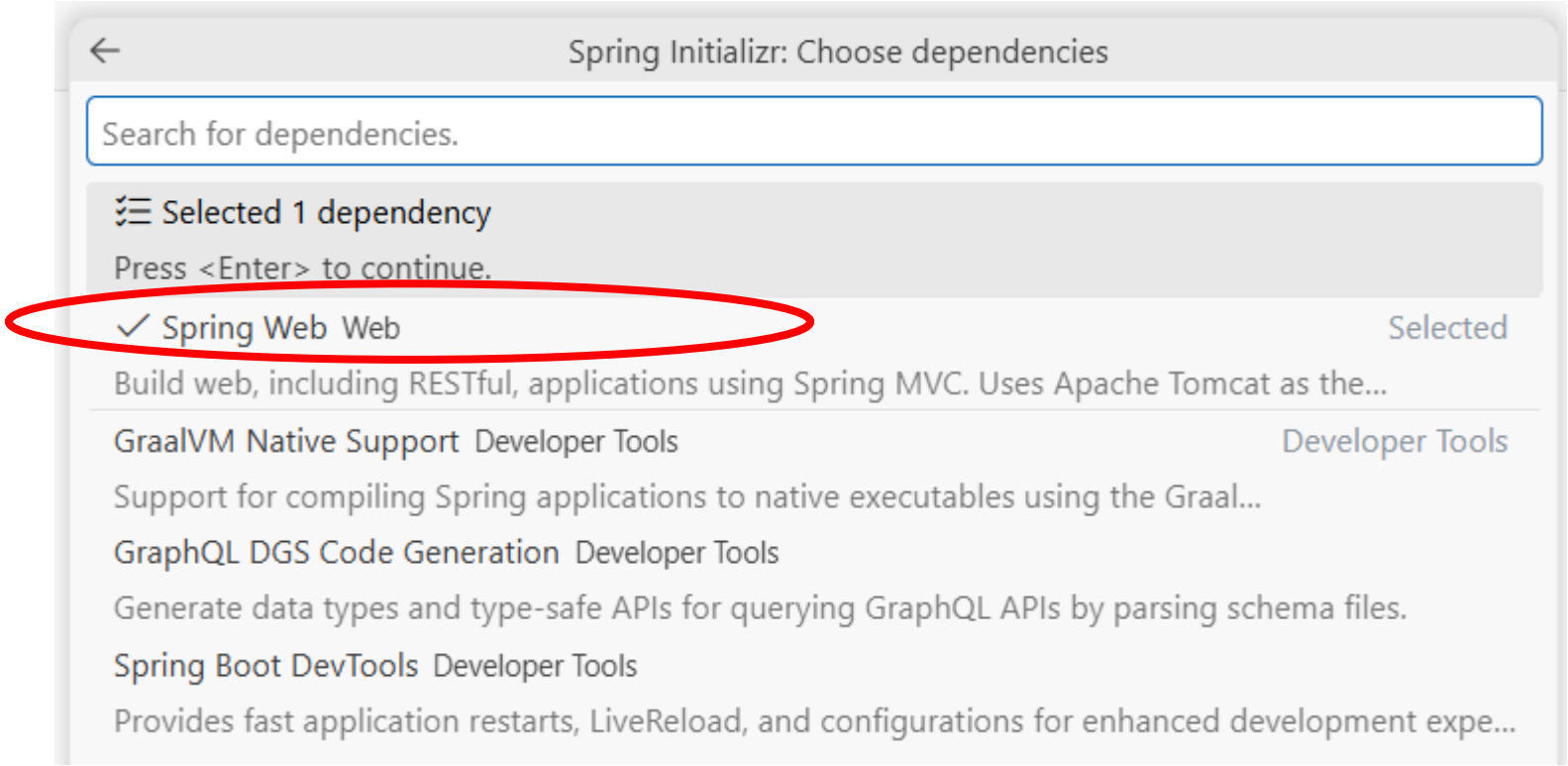


Característica	JAR (Spring Boot)	WAR
Servidor Web	Embutido (Incluso no pacote)	Externo (Instalado no SO)
Execução	Direta via terminal	Gerenciada pelo servidor
Nuvem/Docker	Recomendado (Cloud Native)	Mais complexo de configurar
Estrutura	Focada em classes e libs	Possui pasta `WEB-INF` obrigatória

Versão Java e dependências



Obs.: usar 17



Exemplo 1

File Edit Selection View Go Run Terminal Help projeto1

EXPLORER

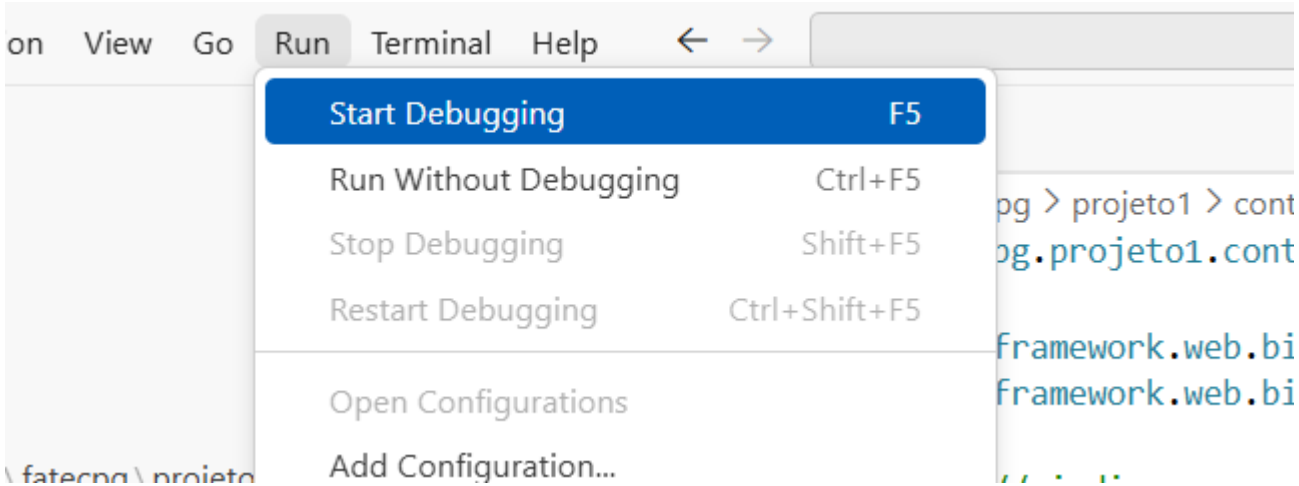
- PROJETO1
  - .mvn
  - .vscode
  - src
    - main
      - java \ com \ fatecpg \ projeto1
        - controller
          - OlaController.java
        - Projeto1Application.java
      - resources
    - test
  - target
  - .gitattributes
  - .gitignore

OlaController.java

src > main > java > com > fatecpg > projeto1 > controller > OlaController.java > ...

```java
1  package com.fatecpg.projeto1.controller;
2
3  import org.springframework.web.bind.annotation.GetMapping;
4  import org.springframework.web.bind.annotation.RestController;
5
6  @RestController  // indica que esta classe é um controlador REST
7  public class OlaController {
8
9      @GetMapping("/ola" )   // mapeia a URL /ola para este método
10     public String ola() {
11         return "Olá, mundo!";
12     }
13
14 }
15
```

Testar

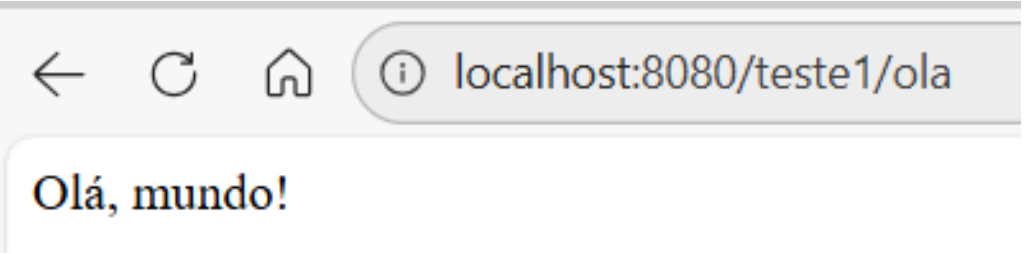


Abrir navegador gerar a requisição a seguir:

localhost:8080/ola

Prefixo das URLs

```java
1   package com.fatecpg.projeto1.controller;
2
3   import org.springframework.web.bind.annotation.GetMapping;
4   import org.springframework.web.bind.annotation.RequestMapping;
5   import org.springframework.web.bind.annotation.RestController;
6
7   @RestController  // indica que esta classe é um controlador REST
8   @RequestMapping("/teste1")  // mapeia todas as URLs que começam com /teste1 para esta classe
9   public class OlaController {
10
11      @GetMapping("/ola" )   // mapeia a URL /ola para este método
12      public String ola() {
13          return "Olá, mundo!";
14      }
15
16  }
```



Parâmetros de rotas

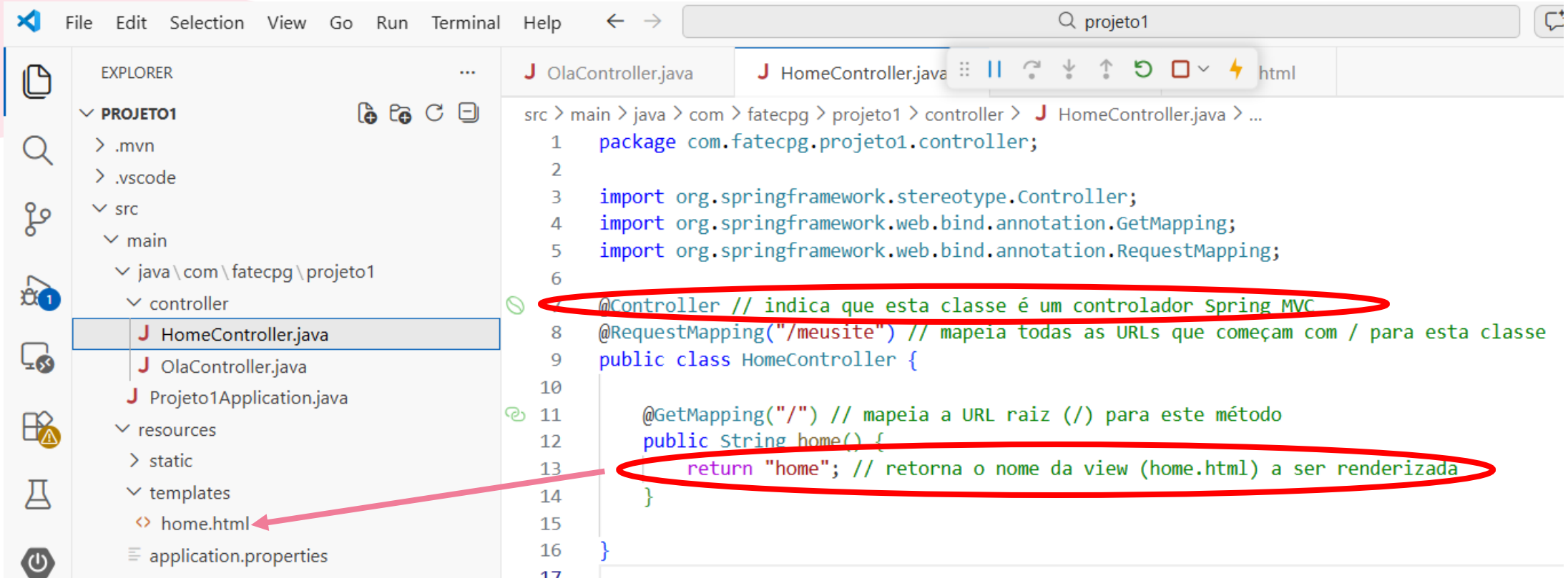
localhost:8080/teste1/soma/3/4

Estilos de formatação

7

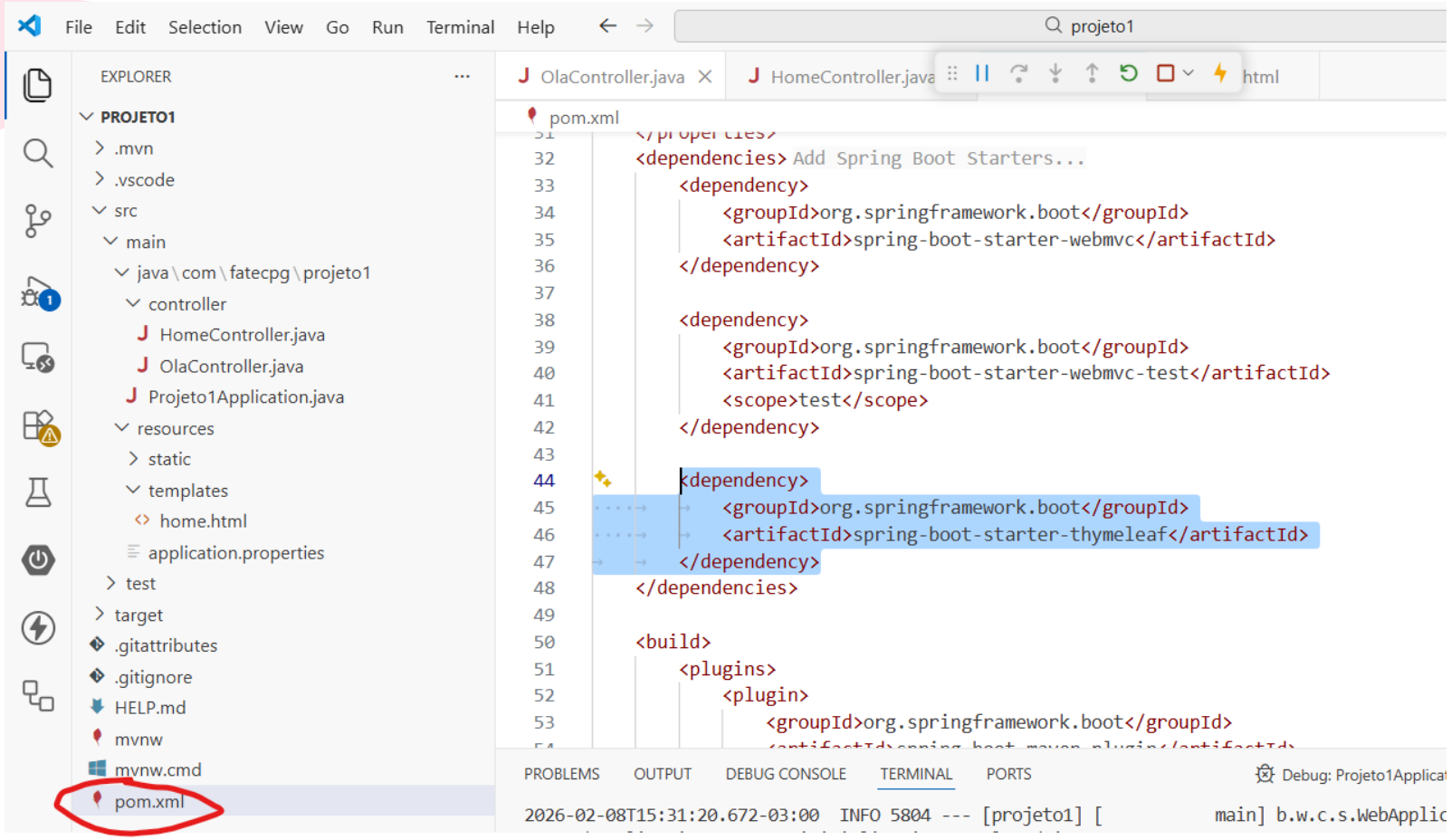
```java
package com.fatecpg.projeto1.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController  // indica que esta classe é um controlador REST
@RequestMapping("/teste1")  // mapeia todas as URLs que começam com /teste1 para esta classe
public class OlaController {

    @GetMapping("/ola" )   // mapeia a URL /ola para este método
    public String ola() {
        return "Olá, mundo!";
    }

    @GetMapping("/soma/{a}/{b}")
    public long calcsoma(@PathVariable long a, @PathVariable long b) {
        return a + b;
    }

}
```

Controller MVC



IMPORTANTE: Views em aplicações MVC são responsáveis por gerar o HTML da resposta à requisição do cliente. No Spring Boot é usado o Thymeleaf que precisa ter dependência adicionada, pois não a selecionamos na criação do projeto.

Adicionar dependência Thymeleaf




```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

View Home

```
<!DOCTYPE html>
<html
xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Home</title>
</head>
<body>
    <h1>Home</h1>
    <p>Bem-vindo</p>
</body>
</html>
```

Recerber o nome como parâmetro da URL

```
@GetMapping("/{nome}") // mapeia a URL raiz (/) para este método
    public String home(@PathVariable String nome) {
        if(nome.isBlank()) {
            nome = "Visitante";
        }
        return "home"; // view (home.html)
    }
```

PROBLEMA: o nome é enviado pelo cliente e precisamos passar para a VIEW inclui-lo no conteúdo HTML que ela gera.

Utiliza-se a inteface Model para isso, conforme exemplificado no próximo slide.

ATENÇÃO: import org.springframework.ui.Model;

Passando dados para a View - Model

```
// mapeia a URL raiz (/) para este método
@GetMapping("/{nome}")
public String home(@PathVariable String nome, Model model) {
    // adiciona o nome do usuário ao modelo para ser usado na view
    model.addAttribute("nomeUsuario", nome);
    return "home"; // retorna o nome da view (home.html) a ser renderizada
}
```

Inserindo nome no HTML - View

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Home</title>
</head>
<body>
    <h1>Home</h1>
    <p>Bem-vindo <span th:text="${nomeUsuario}"></span></p>
</body>
</html>
```